

CSC0056 Data Communication, Homework 4

Department of Computer Science and Information Engineering
National Taiwan Normal University

Nov. 17, 2024

In this assignment, we will again use our Mosquitto codebase. We will empirically validate the claim that the aggregation of slow data traffics will often behave as a Poisson process. In particular, we will see that in the context of broker-based data communication, with many data publishers, each publishing data *at random time points*, the overall interarrival times we observe at the broker will follow an *exponential distribution*.

First, use the following bash shell script to create a stream of uniform-distribution data publishing:

```
1 #!/bin/bash
2 # name this script pub.sh
3 while true;
4 do
5     ./client /mosquitto_pub -h localhost -p 3456 -t topic -m hi 2> /dev/null
6     sleep 0.$(( $RANDOM % 99 + 1 ))
7 done
```

Note that for our purpose here we do not need to run any subscriber.

Now, at your local Mosquitto broker, take a timestamp for each arrival of the PUBLISH message, and then compute the difference between each consecutive pair of timestamps to get the interarrival times. Review the Week-4-1 video on how to do time measurement ([click here for the video link](#)).

Find a good place in the broker code to take a timestamp for each PUBLISH message, and make sure you do not accidentally take timestamps for some other types of control messages. To parse the timestamps, it might help to turn off the default logging messages by setting `log_type none` in `mosquitto.conf`.

Typically, we will use some scripts to help us automatically run the experiments. You can write your own scripts to do the job, and here is just an example written in the bash shell script:

```
1 #!/bin/bash
2 if [[ "$#" -ne 1 ]]; then
3     echo "Error: need to give one parameter!"
4     echo "Usage: $0 #-of-data-streams"
5     echo "Example: $0 100"
```

```

6      exit 2
7  fi
8
9  N=$1
10
11  # Starting the Mosquitto broker
12  ./src/mosquitto -c ./mosquitto.conf 2> tmp &
13  sleep 2
14
15  # Starting publishers
16  for i in $(seq 1 $N); do
17      ./pub.sh &
18      sleep 0.1
19  done
20  echo "Finished starting all publishers"
21  # Keep collecting data
22  sleep 500
23  # Killing all publishers
24  ./kill_pubs.sh
25
26  sleep 10
27
28  # Killing the broker
29  ./kill_brokers.sh
30
31  sleep 2
32
33  # Parsing data
34  awk 'NR==1 {t_prev=$1}\
35      NR > 1 {print ($1-t_prev); t_prev=$1}' ./tmp > interarrivals.out

```

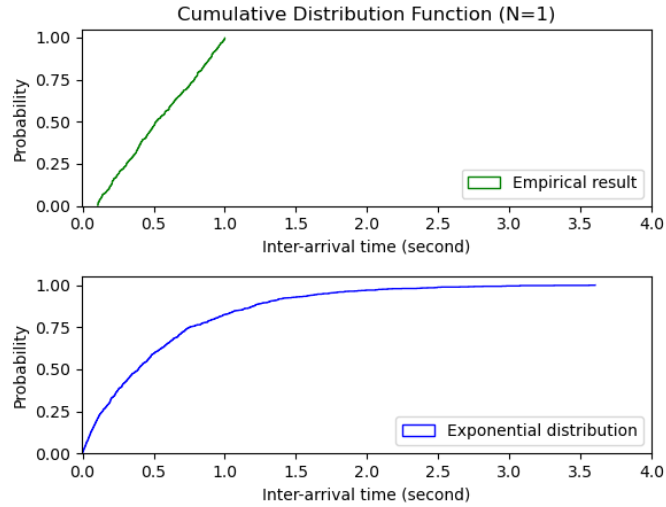
This script does the following in sequence:

1. Start a Mosquitto broker.
2. Start N data streams.
3. Have those N streams send data items to the broker for 500 seconds.
4. Kill all publishers and the broker
5. Finally, parse the timing information and compute the interarrival times seen by the broker.

You may find the related helper scripts by the end of this document.

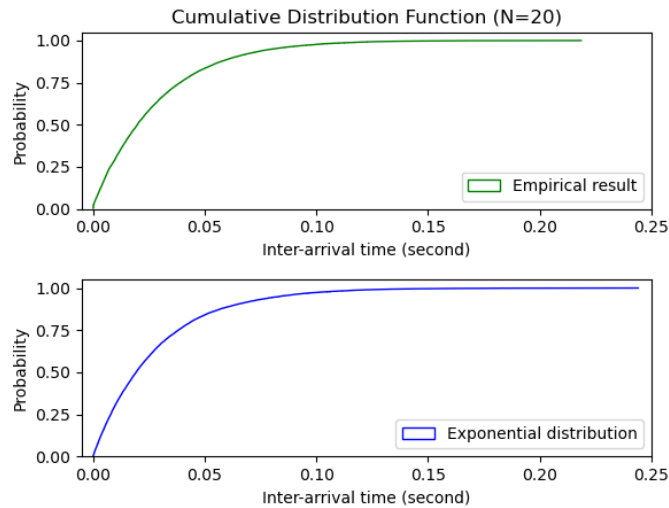
Your task (100 points). Use the publishing script listed on the first page and run N such data streams. Plot the c.d.f. of the interarrival times for $N = 1$, $N = 5$, $N = 10$, $N = 15$, and $N = 20$, respectively, and compare each case with the corresponding exponential distribution. Use the plotting script by the end of this document and submit the figures you plotted (10 points each). And submit your modification on the broker codebase (50 points). Again, submit it as a patch against the commit 84a2250 (review Homework 2 for the relevant instruction).

For example, for $N = 1$, the result should look like the following:



The c.d.f. of the empirical result implies a uniform distribution, which corresponds to our setup in 'pub.sh'.

Now, for $N = 20$, you may observe the following result, which beautifully shows that the distribution of the inter-arrival times indeed approximates the exponential distribution :)



To plot the right exponential distribution for comparison, you need to pay careful attention to the parameter setting. If you will use NumPy+Matplotlib as I

did, in the exponential function you need to both set the right ‘size’ parameter and the right ‘scale’ parameter (click here for the reference):

- The ‘size’ parameter should be set equal to the number of measured interarrival times.
- The ‘scale’ parameter is the reciprocal of the rate of the exponential distribution. Think about how to set this value based on the measurements of interarrival times, so that we may have a fair comparison between the exponential distribution and the empirical result. Script ‘./avg.sh’ can be handy here, which calculates the average value of the given numbers.

Be sure to set the range of x-axis to help clarify your comparison. For example, notice that in the c.d.f. figure above, I purposefully set the x-axis limit the same for both subplots; otherwise, it may not be obvious that the two curves have similar trend. It is easy to set the range of either axis. See line 18 in ‘cdf.py’ for example.

Appendix

Miscellaneous note

- For many of the tools we used in this homework assignment, you may find related tutorials in a MIT online course: <https://missing.csail.mit.edu/>
- When running experiments, it could be helpful to use the `top` utility to help you understand the current workload of the system. You may also run the script ‘kill_pubs.sh’ to help killing some misbehaving Mosquitto publishers.
- For any questions/suggestions regarding the use of bash scripts, awk scripts, python scripts, and cscope, please do not hesitate to post your comments on the course forum in Moodle. Often, people will have similar questions, and we need someone to speak up first.

The related scripts for this homework assignment

The ‘kill_pubs.sh’:

```
1 #!/bin/bash
2 echo "Killing all publishers"
3 killall pub.sh
4 killall mosquitto_pub
```

The ‘kill_brokers.sh’:

```

1 #!/bin/bash
2 # Killing the broker
3 # Note that this would also kill all other
4 # existing mosquitto brokers ran by the same user
5 echo "Killing brokers"
6 killall mosquitto

```

The helper script ‘avg.sh’:

```

1 #!/bin/bash
2 awk 'BEGIN {sum=0} {sum+=$1} END {printf "%.6f\n", sum/NR}'

```

Finally, the plotting python script ‘cdf.py’:

```

1 # This example is derived from the following webpage:
2 # https://matplotlib.org/3.1.1/gallery/statistics/histogram_cumulative.html
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 fig, (ax1, ax2) = plt.subplots(2, 1)
8 fig.subplots_adjust(hspace=0.4)
9
10 # First, plot the empirical result
11 y = np.loadtxt('./interarrivals.out')
12 n_bins = len(y)-1
13 n, bins, patches = ax1.hist(y/1000000, n_bins, density=True,
14     histtype='step', cumulative=True, label='Empirical result',
15     color='g')
16 # the following line is used to remove the last point
17 patches[0].set_xy(patches[0].get_xy()[:-1])
18 ax1.set_xlim(-0.005, 0.12)
19 ax1.set_xlabel('Inter-arrival time (second)')
20 ax1.set_ylabel('Probability')
21 ax1.set_title('Cumulative Distribution Function (N=100)')
22 ax1.legend(loc=4)
23
24 # Then plot the theoretical exponential distribution
25 G = np.random.default_rng()
26 y = G.exponential(scale=0.005523, size=92380)
27 # Now, plot the cumulative distribution function (CDF)
28 n_bins = len(y)-1
29 n, bins, patches = ax2.hist(y, n_bins, density=True, histtype='step',
30     cumulative=True, label='Exponential distribution', color='b')
31 # the following line is used to remove the last point
32 patches[0].set_xy(patches[0].get_xy()[:-1])
33 ax2.set_xlim(-0.005, 0.12)
34 ax2.set_xlabel('Inter-arrival time (second)')
35 ax2.set_ylabel('Probability')
36 #ax2.set_title('Cumulative Distribution Function (N=100)')
37 ax2.legend(loc=4)
38 plt.show()
39 #plt.savefig('./cdf.pdf')

```

Enjoy experimenting :)